

The String class is the undisputed king of Java. While it is technically a Reference Variable (an Object), you will use it so often that Java treats it with special privileges, making it feel almost like a primitive.

1. The Dual Nature of Strings

A String is an array of characters (char) wrapped inside a full Java Object.

Because it is an Object, it lives on the Heap memory, and your variable is just a remote control (reference) pointing to it.

However, because developers use text constantly, Java provides a shortcut called a **String Literal**.

Java

```
// 1. The Standard Object Way (Rarely used)
String agencyName = new String("Kodelix");

// 2. The Literal Shortcut (The standard way)
String agencyName = "Kodelix";
```

2. The String Constant Pool (Memory Magic)

If an enterprise application creates the word "Active" 10,000 times for user statuses, creating 10,000 separate Objects on the Heap would destroy memory.

To solve this, the JVM carves out a special, highly optimized area on the Heap called the **String Pool**.

- When you create a String using literal quotes ("Kodelix"), Java checks the Pool.
- If "Kodelix" already exists in the Pool, Java does *not* create a new object. It simply points your new reference variable to the exact same object.
- If you use the new keyword, you force Java to bypass the Pool and build a brand-new, isolated object out on the general Heap.

3. The Golden Rule of Immutability

A String object can never be changed once it is created. This is called being *Immutable*.

If you try to modify a String, the JVM does not change the original object. Instead, it creates a brand-new object with the modified text, points your variable to the new object, and leaves the old one behind for Garbage Collection.

Why are Strings Immutable?

1. **Security:** Strings hold sensitive data (database URLs, passwords, network ports). If a malicious method could change a String's value after a security check passed, the system could be compromised.
2. **Thread Safety:** Because the data can never change, thousands of separate threads can read the same String object simultaneously without breaking each other's logic.
3. **The String Pool:** The Pool only works *because* Strings are immutable. If var1 and var2 point to the same "Kodelix" object, and var1 was allowed to change it to "Kodelix_Pro", it would secretly break var2.

4. Comparison: == vs .equals()

This is the most common bug in junior developer code.

- **== (The Identity Operator):** Compares the **memory addresses**. It asks: *"Are these two remote controls pointing to the exact same physical TV?"*
- **.equals() (The Value Operator):** Compares the **actual text inside**. It asks: *"Do these two TVs display the exact same picture?"*

Always, without exception, use .equals() when comparing Strings.

Interview Vault: The Viral & Tricky String Questions

This section contains the most heavily tested edge cases in modern Java interviews.

Q: "What will the following code print? Explain exactly why."

Java

```
String s = "ClearFlow";  
s.concat(" App");  
System.out.println(s);
```

Answer: It prints **"ClearFlow"** (not "ClearFlow App").

The Trap: This tests Immutability. The .concat() method does not change the original object. It creates a brand-new object containing "ClearFlow App". However, because the developer did not assign that new object to a variable (e.g., `s = s.concat(" App");`), the new object is immediately lost in memory, and s continues to point to the original, unchanged "ClearFlow" object.

Q: "Predict the output of the following math and string concatenations."

Java

```
System.out.println(10 + 20 + "Java");  
System.out.println("Java" + 10 + 20);
```

Answer: It prints **30Java** and then **Java1020**.

The Trap: Java evaluates expressions from left to right.

- Line 1: 10 + 20 is evaluated first as standard math (30). Then it encounters a String, so it converts the 30 to text: "30Java".
- Line 2: It starts with a String "Java". Once a String is encountered, everything after it is treated as text concatenation. "Java" + 10 becomes "Java10". Then "Java10" + 20 becomes "Java1020".

Q: "Explain what the .intern() method does."

Java

```
String s1 = new String("Admin");  
String s2 = "Admin";  
String s3 = s1.intern();  
  
System.out.println(s1 == s2);  
System.out.println(s2 == s3);
```

Answer: It prints **false** then **true**.

The Trap: s1 is created with new, so it lives outside the String Pool. s2 is a literal, so it lives inside the Pool. Therefore, s1 == s2 is false.

The .intern() method is a manual command that tells the JVM: *"Take the text inside this object, check if it exists in the String Pool. If it does, return the memory address of the Pool version."* Therefore, s3 receives the address of the pooled "Admin", making it point to the exact same object as s2.

Q: "For highly sensitive security data like a user's password, why is it considered a critical vulnerability to store it in a String? What should you use instead?"

Answer: You should use a char[] (character array) instead of a String.

The Trap: Because Strings are immutable and heavily cached in the String Pool, they cannot be erased. If you store a password in a String, that plaintext password will float around in the system's RAM for an unpredictable amount of time until the Garbage Collector finally deletes it. A hacker taking a memory dump of the server can easily extract it.

If you use a `char[]`, you can manually overwrite the array with dummy data (e.g., `passwordArray[0] = '*';`) the exact millisecond you are done authenticating the user, instantly wiping the password from RAM.

Q: "If you need to reverse a String or build a massive String in a loop, why shouldn't you use a for loop and the += operator?"

Java

```
String result = "";
for(int i=0; i<1000; i++) {
    result += "a";
}
```

Answer: Because of Immutability, `result += "a"` forces the JVM to create a brand-new object on every single iteration of the loop, resulting in 1,000 abandoned objects cluttering the Heap. This causes a massive performance drop due to Garbage Collection overhead. You must use `StringBuilder` (or `StringBuffer` if thread-safe) because it modifies a single, mutable object without creating new ones.